



High-Throughput Ring Buffers

Lesson 06: Streaming High-Frequency Audit Logs to User-Space Without Loss

Table of Contents

Lesson 06: High-Throughput Ring Buffers

1. Lesson Overview

2. Foundations & Core Technical Concepts

The Problem with `BPF_MAP_TYPE_PERF_EVENT_ARRAY`

What is `BPF_MAP_TYPE_RINGBUF`?

How the Lock-free Reservation Works

`bpf_ringbuf_reserve` vs `bpf_ringbuf_output`

Event Loss and the `BPF_RB_NO_WAKEUP` Flag

3. Problem Statement & Real-World Use Case

The Challenge: Streaming 1M+ Events/Second

4. Perf Buffer vs Ring Buffer — Deep Comparison

5. Internal Kernel Flow

6. Architecture Diagram

7. Event Lifecycle Sequence

8. Ring Buffer Memory Layout

9. Step-by-Step Code Walkthrough

The eBPF C Program (`main.bpf.c`)

The Go Loader (`main.go`)

10. Running the Lab

Phase 1: Compile

Phase 2: Generate File Open Events

Phase 3: Observe Output & Drop Rate

11. Debugging

12. Verifier Errors & Common Mistakes

Error: ringbuf size must be power of 2

Common Mistake: Not checking `bpf_ringbuf_reserve` return value

Common Mistake: Using `bpf_ringbuf_discard` after writing

13. Performance Analysis

14. Best Practices

15. Summary

16. Further Reading

Technical Deep-Dive Q&A: BPF Ring Buffer

1. Beginner Level

Q1. What problem does BPF Ring Buffer solve that Perf Event Array cannot?

Q2. What does "zero-copy" mean in the context of BPF Ring Buffer writes?

2. Intermediate Level

Q3. Explain the three-step Ring Buffer write protocol: reserve, fill, commit.

Q4. How does the Ring Buffer notify user-space? What is the role of epoll?

Q5. What happens when the Ring Buffer is full and a new event cannot be reserved?

3. Advanced Level

Q6. Why does Ring Buffer guarantee event ordering while Perf Event Array does not?

4. System Design

Q7. Design a file integrity monitor using Ring Buffer that detects modifications to /etc/ files in real time across a 1000-node Kubernetes cluster.

5. Troubleshooting

Q8. Your Ring Buffer consumer goroutine reports "ring buffer full" drops even though your buffer is 16MB. What are 3 causes?

Hands-on Lab & Homework Assignments

1. Practical Exercises

Task 1: Compare Perf Buffer vs Ring Buffer Memory Usage

Task 2: Measure Drop Rate Under Load

2. Coding Assignment: Drop Counter Map

3. Mini-Project: File Integrity Monitor

4. Advanced Challenge: Throughput Benchmark

Lesson 06: High-Throughput Ring Buffers

1. Lesson Overview

In this lesson, you will build a **high-throughput kernel event streamer** using the modern **BPF Ring Buffer** (`BPF_MAP_TYPE_RINGBUF`), replacing the older Perf Event Array with a superior zero-copy, single-buffer design. We will trace every `openat()` syscall on the host, stream structured log events to user-space at rates exceeding 1 million events/second, and implement a production-grade event loss detection system.

This is the same data pipeline architecture used by Falco, Sysdig, and Tracee for their kernel event streaming engines.

2. Foundations & Core Technical Concepts

The Problem with `BPF_MAP_TYPE_PERF_EVENT_ARRAY`

The Perf Event Array (used in Series 02) was the original high-throughput streaming mechanism. It works but has significant limitations at scale:

Problem 1 — Per-CPU Memory Waste: Each CPU core gets its own separate ring buffer allocation. A 64-core server with 1MB per-CPU buffers wastes 64MB of memory, even if most cores are idle. If CPU 0 handles 90% of the load, its buffer overflows while CPU 63's buffer sits empty.

Problem 2 — Out-of-Order Events: Since each CPU has an independent buffer, events from different CPUs arrive in the user-space reader in CPU-completion order, not wall-clock order. If CPU 0 generates event A at $T=1000\text{ns}$, and CPU 1 generates event B at $T=1001\text{ns}$, they might arrive as B then A.

Problem 3 — Wakeup Overhead: Each event submission potentially wakes up the user-space reader via a `perf_event` notification — expensive for high-frequency events.

What is `BPF_MAP_TYPE_RINGBUF`?

`BPF_MAP_TYPE_RINGBUF` (introduced in Linux 5.8) is a **single, shared, lock-free ring buffer** used by all CPU cores simultaneously.

- **Single Buffer:** One contiguous memory allocation shared across all CPUs — no per-CPU waste.
- **Lock-free Reservation:** Uses a sophisticated atomic reservation protocol. CPU cores race to reserve space in the buffer using atomic compare-and-swap operations, with no kernel locks.
- **Zero-copy Commit:** The BPF program writes directly into the shared ring buffer memory (not onto the BPF stack first). No memcpy from stack to buffer.
- **Ordered Delivery:** Events are delivered to user-space in the order they were **committed** to the ring buffer.

How the Lock-free Reservation Works

The ring buffer maintains two pointers:

- **Producer position** (tail): Where the next write goes.
- **Consumer position** (head): Where the next read comes from.

When a BPF program calls `bpf_ringbuf_reserve(size)`:

1. The kernel atomically increments the producer tail by `size` using `xadd` (atomic add).
2. The BPF program gets back a pointer to the reserved region.
3. The BPF program writes data directly into that memory region.
4. `bpf_ringbuf_submit()` marks the region as readable.

This `reserve` → `write` → `submit` protocol means **multiple CPU cores can be writing simultaneously** without any lock contention.

`bpf_ringbuf_reserve` vs `bpf_ringbuf_output`

Two ways to write to a ring buffer:

```
// Method 1: Reserve → write → submit (ZERO COPY — preferred)
struct event *e = bpf_ringbuf_reserve(&rb, sizeof(*e), 0);
if (!e) return 0;    // Buffer full — event dropped
e->pid = 1234;        // Write directly into ring buffer memory
e->len = 42;
bpf_ringbuf_submit(e, 0);    // Commit to reader

// Method 2: Output (copies from BPF stack — avoid for large structs)
struct event e = {};
e.pid = 1234;
bpf_ringbuf_output(&rb, &e, sizeof(e), 0); // One extra memcpy
```

Event Loss and the BPF_RB_NO_WAKEUP Flag

For extremely high event rates, you can batch notifications:

```
// BPF_RB_NO_WAKEUP — don't wake up user-space after this submit
// Use for high-frequency events; user-space will poll instead
bpf_ringbuf_submit(e, BPF_RB_NO_WAKEUP);

// BPF_RB_FORCE_WAKEUP — always wake up user-space immediately
bpf_ringbuf_submit(e, BPF_RB_FORCE_WAKEUP);
```

3. Problem Statement & Real-World Use Case

The Challenge: Streaming 1M+ Events/Second

Sysdig (now Falco's parent company) built their container security platform on top of a custom kernel driver that exported kernel events to user-space. At scale (100-core hosts, 50k containers), this driver was processing 5+ million syscall events per second.

When Sysdig migrated to eBPF:

- The Perf Event Array design dropped events under load because per-CPU buffers were too small.
- Increasing buffer sizes wasted gigabytes of memory across a 1000-node Kubernetes cluster.
- The Ring Buffer's single shared allocation solved both problems simultaneously.

Tracee (Aqua Security's open-source eBPF security tool) switched from Perf Event Array to Ring Buffer in version 0.8 and reported:

- **50% reduction in memory usage** at the same event rate.
- **Near-zero event drops** under load that previously caused 5–10% loss.
- **Consistent event ordering** critical for attack chain reconstruction.

Falco uses a similar architecture for their kernel event streaming pipeline, processing millions of syscall events per second with sub-1ms p99 latency to their rules engine.

4. Perf Buffer vs Ring Buffer — Deep Comparison

Property	BPF_MAP_TYPE_PERF_EVENT_ARRAY	BPF_MAP_TYPE_RINGBUF
Memory Model	Per-CPU separate buffers	Single shared buffer
Memory Efficiency	Wastes memory on idle CPUs	Optimal — dynamically shared
Event Ordering	Per-CPU order only	Global commit order
Wakeup Mechanism	Signal per event	Configurable batching
Zero-copy	No (stack → buffer copy)	Yes (reserve → write in-place)
Kernel Version	4.4+	5.8+
Max Event Size	Page size (4KB)	Up to half ring buffer size
Drop Detection	Via <code>perf_event_lost</code>	Via <code>bpf_ringbuf_reserve</code> NULL

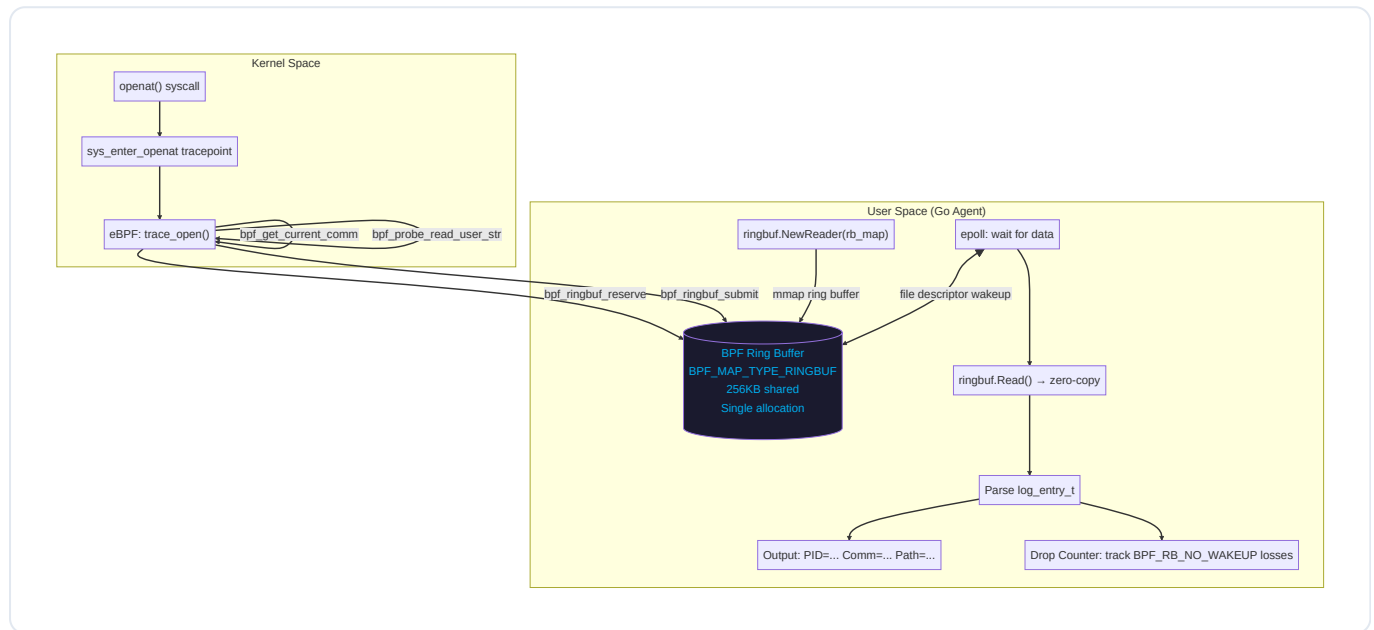
5. Internal Kernel Flow

When a process calls `openat(AT_FDCWD, "/etc/passwd", O_RDONLY)` :

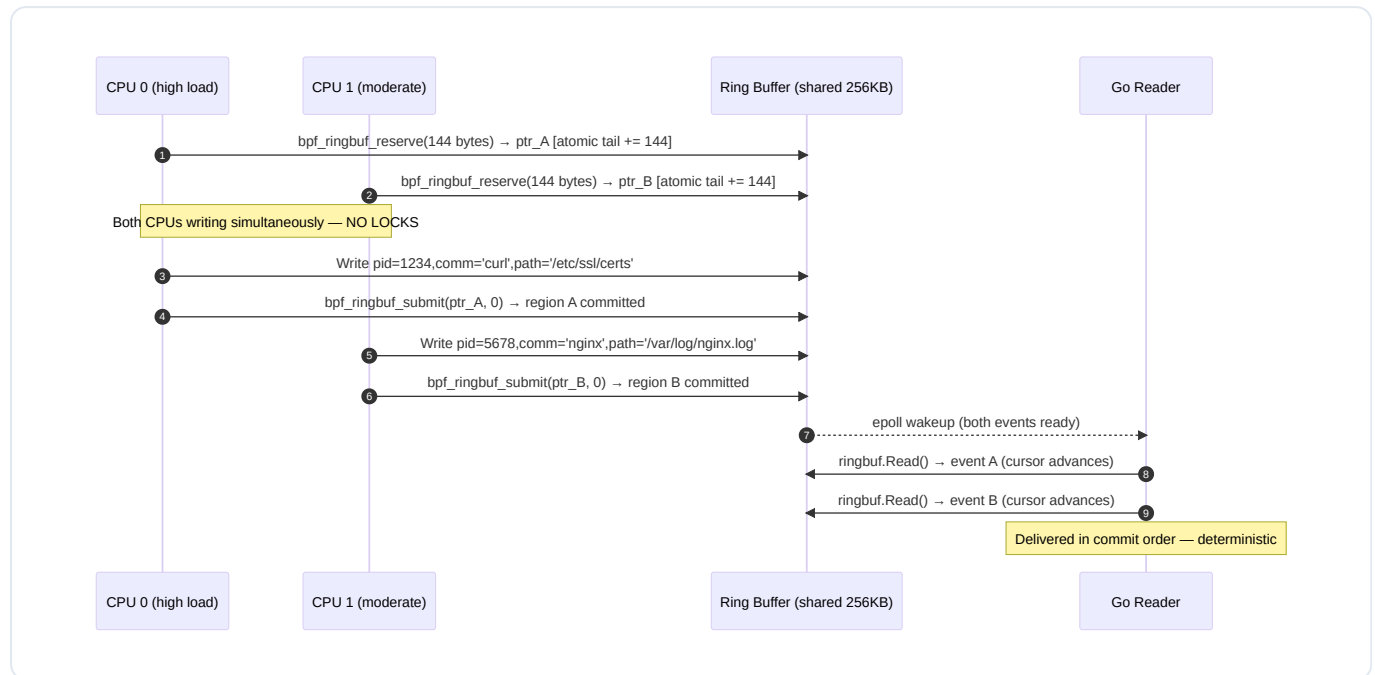
1. The `openat` system call fires the `sys_enter_openat` tracepoint.
2. Our eBPF program calls `bpf_ringbuf_reserve(&rb, sizeof(struct log_entry_t), 0)`.
3. The kernel atomically reserves `sizeof(struct log_entry_t)` bytes in the ring buffer, returning a pointer to that region.
4. Our eBPF program writes PID, command name, and filename directly into that memory.

5. `bpf_ringbuf_submit()` marks the region as committed.
6. If `epoll` is watching the ring buffer FD (user-space), it wakes up.
7. User-space calls `ringbuf.Read()` which maps the ring buffer memory and reads the committed region — no copy from kernel to user space.

6. Architecture Diagram



7. Event Lifecycle Sequence

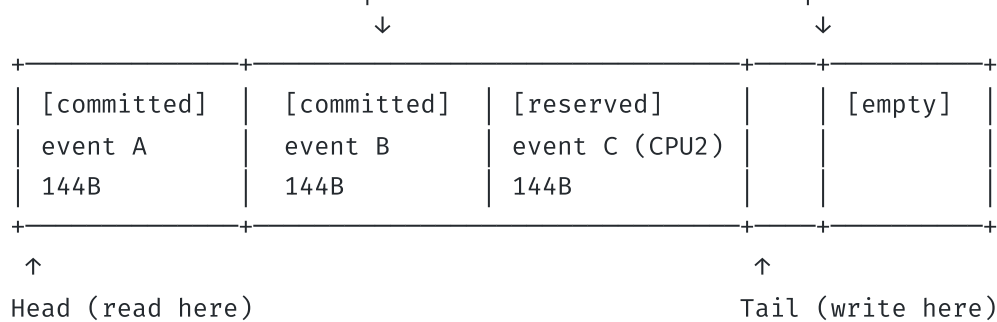


8. Ring Buffer Memory Layout

BPF Ring Buffer: rb (256KB total)

Producer tail

Consumer head



struct log_entry_t layout (144 bytes):



Total: 148B (padded to 152B for alignment)

9. Step-by-Step Code Walkthrough

The eBPF C Program (`main.bpf.c`)

```
#include <vmlinux.h>
#include <bpf/bpf_helpers.h>

struct log_entry_t {
    __u32 pid;
    char comm[16];
    char filename[128];
};

// Single shared ring buffer – 256KB is enough for ~1700 events in-flight
// Size MUST be a power of 2 and a multiple of the page size (4096)
struct {
    __uint(type, BPF_MAP_TYPE_RINGBUF);
    __uint(max_entries, 256 * 1024); // 256KB
} rb SEC(".maps");

// Optional: count how many events were dropped due to full buffer
struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __uint(max_entries, 1);
    __type(key, __u32);
    __type(value, __u64);
} drop_counter SEC(".maps");

SEC("tp/syscalls/sys_enter_openat")
int trace_open(struct trace_event_raw_sys_enter_openat_t *ctx) {
    // Attempt to reserve space directly in the ring buffer
    // This is a ZERO-COPY operation – we write into ring buffer memory directly
    struct log_entry_t *entry = bpf_ringbuf_reserve(&rb, sizeof(*entry), 0);
    if (!entry) {
        // Buffer is full – event will be dropped
        // Increment drop counter so user-space can track loss rate
        __u32 key = 0;
        __u64 *drops = bpf_map_lookup_elem(&drop_counter, &key);
        if (drops) __sync_fetch_and_add(drops, 1);
        return 0;
    }

    // Write fields directly into ring buffer memory (no stack copy)
    entry->pid = bpf_get_current_pid_tgid() >> 32;
```

```
    bpf_get_current_comm(&entry->comm, sizeof(entry->comm));
    bpf_probe_read_user_str(&entry->filename, sizeof(entry->filename),
                           (void *)ctx->filename);

    // Commit – makes this region visible to user-space reader
    // Default flag (0) = wake up epoll if needed
    bpf_ringbuf_submit(entry, 0);
    return 0;
}

char LICENSE[] SEC("license") = "GPL";
```

The Go Loader (`main.go`)

```
package main

import (
    "bytes"
    "encoding/binary"
    "fmt"
    "log"
    "os"
    "os/signal"
    "syscall"
    "time"

    "github.com/cilium/ebpf"
    "github.com/cilium/ebpf/link"
    "github.com/cilium/ebpf/ringbuf"
)

type LogEntry struct {
    PID      uint32
    Comm     [16]byte
    Filename [128]byte
}

func main() {
    spec, err := ebpf.LoadCollectionSpec("main.bpf.o")
    if err != nil {
        log.Fatalf("load spec: %v", err)
    }

    var objs struct {
        Prog      *ebpf.Program `ebpf:"trace_open"`
        Rb        *ebpf.Map     `ebpf:"rb"`
        DropCounter *ebpf.Map     `ebpf:"drop_counter"`
    }
    if err := spec.LoadAndAssign(&objs, nil); err != nil {
        log.Fatalf("load objects: %v", err)
    }
    defer objs.Prog.Close()
    defer objs.Rb.Close()
    defer objs.DropCounter.Close()

    tp, err := link.Tracepoint("syscalls", "sys_enter_openat", objs.Prog, nil)
    if err != nil {
        log.Fatalf("attach tracepoint: %v", err)
    }
}
```

```

}
defer tp.Close()

// ringbuf.NewReader maps the ring buffer into user-space memory
// Reads are zero-copy – the kernel's ring buffer memory is directly accessible
rd, err := ringbuf.NewReader(objs.Rb)
if err != nil {
    log.Fatalf("ringbuf reader: %v", err)
}
defer rd.Close()

fmt.Printf("%-6s %-16s %s\n", "PID", "COMM", "PATH")

// Track drop rate
dropTicker := time.NewTicker(5 * time.Second)
defer dropTicker.Stop()

sig := make(chan os.Signal, 1)
signal.Notify(sig, os.Interrupt, syscall.SIGTERM)
go func() { ←sig; rd.Close() }()

// Monitor drop counter in background
go func() {
    for range dropTicker.C {
        var key uint32
        var drops uint64
        _ = objs.DropCounter.Lookup(&key, &drops)
        if drops > 0 {
            log.Printf("⚠ Ring buffer overflow – %d events dropped in last 5s'
            // Reset counter
            _ = objs.DropCounter.Update(&key, &uint64(0), ebpf.UpdateExist)
        }
    }
}()

var entry LogEntry
for {
    // rd.Read() blocks until an event is committed to the ring buffer
    // Returns the raw bytes of the reserved region – zero copy from kernel
    record, err := rd.Read()
    if err != nil {
        if ringbuf.IsClosed(err) { return }
        continue
    }

    if err := binary.Read(bytes.NewBuffer(record.RawSample), binary.LittleEndian
    continue

```

```

    }

    comm := string(bytes.TrimRight(entry.Comm[:], "\x00"))
    path := string(bytes.TrimRight(entry.Filename[:], "\x00"))
    fmt.Printf("%-6d %-16s %s\n", entry.PID, comm, path)
}
}

```

10. Running the Lab

Phase 1: Compile

```

cd series/series-06/source-code
make
sudo ./loader

```

Phase 2: Generate File Open Events

```

# In another terminal, generate heavy file activity
find / -name "*.conf" 2>/dev/null
cat /proc/1/maps
ls -la /etc/

# Stress test: generate 100k file opens
for i in $(seq 1 100000); do cat /dev/null; done

```

Phase 3: Observe Output & Drop Rate

```

PID    COMM      PATH
1234   find      /etc/nginx/nginx.conf
1235   cat       /proc/1/maps
9876   ls        /etc/
⚠️  Ring buffer overflow – 142 events dropped in last 5s

```

If you see drops, increase the ring buffer size from `256 * 1024` to `1 << 22` (4MB) in the C code.

11. Debugging

```
# Check if ring buffer is mounted and active
sudo bpftool map list | grep ringbuf
sudo bpftool map show id <ID>
# Look for: max_entries 262144 (262144 = 256*1024 bytes)

# Monitor the ring buffer fill rate
sudo bpftool map show id <ID> | grep -i "bytes\|entries"
```

12. Verifier Errors & Common Mistakes

Error: `ringbuf size must be power of 2`

```
invalid BPF_MAP_TYPE_RINGBUF: max_entries must be a power of 2
```

Cause: The ring buffer `max_entries` must be a power of 2 AND a multiple of `PAGE_SIZE` (4096).

Fix: Use values like `4096`, `8192`, `65536`, `131072`, `262144` (256KB), `1048576` (1MB).

Common Mistake: Not checking `bpf_ringbuf_reserve` return value

```
// WRONG - if reserve returns NULL, dereferencing causes a verifier error
struct log_entry_t *e = bpf_ringbuf_reserve(&rb, sizeof(*e), 0);
e->pid = 1234; // Verifier rejects: e could be NULL
```

Fix: Always null-check the return value before writing.

Common Mistake: Using `bpf_ringbuf_discard` after writing

If you reserved space but decide not to submit (e.g., a filter condition fails), you **MUST** call `bpf_ringbuf_discard()` to return the reserved space:

```
struct event *e = bpf_ringbuf_reserve(&rb, sizeof(*e), 0);
if (!e) return 0;
if (should_skip) {
    bpf_ringbuf_discard(e, 0); // Return space to the pool
    return 0;
}
bpf_ringbuf_submit(e, 0);
```

13. Performance Analysis

Metric	Perf Event Array	Ring Buffer
Memory per-core (64-core, 1MB each)	64MB	Shared — configurable
Event ordering	Per-CPU only	Global commit order
Zero-copy write	No	Yes
Kernel version required	4.4	5.8
Events/sec before drops	~500K/core	~1M+ shared

14. Best Practices

- **Size the ring buffer based on burst capacity**, not average rate. A 1ms burst of 10,000 events at 1KB each needs 10MB of ring buffer.
- Use `BPF_RB_NO_WAKEUP` **for bulk processing**: Batch multiple events before waking user-space to reduce context switch overhead.
- **Monitor the drop counter**: A drop counter is not optional in production — it's the only signal that your ring buffer is undersized.

15. Summary

- `BPF_MAP_TYPE_RINGBUF` replaces Perf Event Array for high-throughput event streaming on kernels 5.8+.
 - A single shared buffer eliminates per-CPU memory waste and ensures global event ordering.
 - `bpf_ringbuf_reserve()` writes zero-copy directly into ring buffer memory.
 - The reserve → write → submit protocol is lock-free using atomic operations.
 - Always implement drop detection with a counter map.
-

16. Further Reading

- **BPF Ring Buffer Design:** [Andrii Nakryiko — BPF Ring Buffer](#)
- **Kernel Ring Buffer Documentation:** [Linux BPF Ring Buffer Docs](#)
- **Tracee Architecture:** [Aqua Tracee eBPF Streaming](#)
- **Falco Kernel Event Streaming:** [Falco eBPF Driver Architecture](#)

Technical Deep-Dive Q&A: BPF Ring Buffer

1. Beginner Level

Q1. What problem does BPF Ring Buffer solve that Perf Event Array cannot?

Answer: `BPF_MAP_TYPE_PERF_EVENT_ARRAY` allocates a **separate** ring buffer per CPU core. On a 64-core server with 1MB per-CPU allocation that is 64MB of memory — even when 63 cores are idle. Events from different CPUs arrive **out of chronological order** in user-space.

`BPF_MAP_TYPE_RINGBUF` (Linux 5.8+) uses a **single shared buffer** for all CPUs. CPUs write using a lock-free atomic reservation (`compare-and-swap`). Events are delivered in **commit order** — globally consistent. Memory is configurable (e.g., 4MB total regardless of CPU count).

Q2. What does "zero-copy" mean in the context of BPF Ring Buffer writes?

Answer: With Perf Event Array, `bpf_perf_event_output()` copies event data from the BPF stack into the ring buffer — two separate memory regions.

With Ring Buffer, `bpf_ringbuf_reserve(size)` returns a **direct pointer into the ring buffer memory**. You write your event fields into that pointer. No copy occurs — your program writes directly into the shared-memory region that user-space reads from. This eliminates one memcopy per event, saving ~10ns per event at 1M events/sec = 10ms/sec of pure copy overhead avoided.

2. Intermediate Level

Q3. Explain the three-step Ring Buffer write protocol: reserve, fill, commit.

Answer:

```
// Step 1: Reserve space – atomic CAS on the producer tail pointer
struct event_t *e = bpf_ringbuf_reserve(&rb, sizeof(*e), 0);
if (!e) return 0; // Buffer full – drop event (increment drop counter)

// Step 2: Fill directly – writing into ring buffer memory (zero-copy)
e->pid = bpf_get_current_pid_tgid() >> 32;
bpf_get_current_comm(&e->comm, sizeof(e->comm));
bpf_probe_read_user_str(&e->filename, sizeof(e->filename), fname);

// Step 3: Commit – advances the consumer-visible tail, wakes epoll
bpf_ringbuf_submit(e, 0);
// Alternative: bpf_ringbuf_discard(e, 0) – cancels without publishing
```

The `reserve` is atomic and lock-free. Multiple CPUs can reserve simultaneously — each gets a unique slot. Commits can happen out-of-reservation-order; the kernel reorders them for consumer visibility.

Q4. How does the Ring Buffer notify user-space? What is the role of epoll?

Answer: After `bpf_ringbuf_submit()`, the kernel checks if user-space is blocked in `epoll_wait()` on the ring buffer's file descriptor. If yes, it sends a wakeup. The `cilium/ebpf ringbuf.Reader.Read()` uses `epoll` internally — it blocks with zero CPU usage until data arrives, then wakes and reads all available records in a batch.

This is vastly more efficient than polling: at 0 events/sec the reader uses 0% CPU. At 1M events/sec it wakes up, drains the buffer, and blocks again — typically one `epoll` wakeup per batch of hundreds of events.

Q5. What happens when the Ring Buffer is full and a new event cannot be reserved?

Answer: `bpf_ringbuf_reserve()` returns `NULL`. The eBPF program receives a null pointer and **must** handle it — typically by returning 0 immediately (dropping the event). The kernel does NOT block or retry.

User-space can detect drops by reading `ring_buffer__poll()` statistics or by maintaining a drop counter in a separate BPF map that the eBPF program increments when `reserve` returns `NULL`.

To reduce drops: increase the ring buffer size (must be a power of 2, max 2GB), or reduce the event struct size, or filter events in kernel before submitting.

3. Advanced Level

Q6. Why does Ring Buffer guarantee event ordering while Perf Event Array does not?

Answer: Perf Event Array has **N independent buffers** (one per CPU). Each CPU writes to its own buffer. User-space reads all N buffers round-robin, but has no way to know which event happened first across CPUs without an embedded timestamp.

Ring Buffer has **one shared buffer**. All CPUs compete for slots via atomic CAS on the producer write pointer. The order in which CPUs win the CAS determines the order of events in the buffer. Events committed earlier have lower buffer offsets → user-space reads them in offset order → global commit ordering is preserved.

For attack chain reconstruction (`bash spawned curl which opened /etc/passwd`), ordering is correctness-critical. Without it you might see the file open before the process that caused it.

4. System Design

Q7. Design a file integrity monitor using Ring Buffer that detects modifications to /etc/ files in real time across a 1000-node Kubernetes cluster.

Answer:

Per-node DaemonSet:

```
tracepoint/syscalls/sys_enter_openat
→ filter: path starts with /etc/
→ filter: flags & O_WRONLY or O_RDWR
→ ringbuf_reserve → fill {pid, uid, comm, path, timestamp}
→ ringbuf_submit
```

Go Agent:

```
ringbuf.NewReader → streams events
→ check path against allowlist (map lookup)
→ emit as OpenTelemetry log to central collector
→ if unknown writer: send SIGKILL via bpf_send_signal + alert PagerDuty
```

Central Aggregator (Kafka + ClickHouse):

```
→ Store all events for 90-day audit trail
→ Alert on: UID=0 writing to /etc/passwd
```

Key decision: Use Ring Buffer (not Perf Event Array) because `/etc/` write events are rare — shared buffer wastes less memory than 64 per-CPU allocations that are mostly empty.

5. Troubleshooting

Q8. Your Ring Buffer consumer goroutine reports "ring buffer full" drops even though your buffer is 16MB. What are 3 causes?

Answer:

1. **Consumer too slow:** Your Go processing code does heavy work (JSON marshaling, HTTP calls) inside the read loop. The kernel is producing faster than you consume. Fix: decouple reading from processing — read into a Go channel, process in a separate goroutine pool.
2. **Buffer size not a power of 2:** Ring Buffer size MUST be a power of 2 (4MB, 8MB, 16MB, 32MB...). If you specified 16,000,000 (not 16,777,216), the kernel rounds down silently and your effective buffer is smaller than expected.
3. **Event burst from a noisy process:** One process calling `open()` in a tight loop (e.g., a misconfigured logger) floods the buffer. Add a per-PID rate limit map: count events per PID per second and drop in-kernel if exceeded.

Hands-on Lab & Homework Assignments

1. Practical Exercises

Task 1: Compare Perf Buffer vs Ring Buffer Memory Usage

```
# Load perf buffer version (1MB per CPU)
sudo ./loader --mode perf
sudo bpftool map list | grep events
# Note: size = num_cpus × 1MB

# Load ring buffer version
sudo ./loader --mode ring
sudo bpftool map list | grep rb
# Note: single shared allocation
```

On your host: how much memory does Perf Array use vs Ring Buffer for the same event throughput?

Task 2: Measure Drop Rate Under Load

```
# Generate 500K execve events/sec
for i in $(seq 1 500000); do /bin/true; done &
sudo ./loader # watch for "Lost N events" messages
```

Increase ring buffer size from 4MB → 16MB → 64MB. At what size do drops reach zero?

2. Coding Assignment: Drop Counter Map

Add a per-CPU drop counter that increments when `bpf_ringbuf_reserve` returns NULL:

```

struct {
    __uint(type, BPF_MAP_TYPE_PERCPU_ARRAY);
    __uint(max_entries, 1);
    __type(key, __u32);
    __type(value, __u64);
} drop_count SEC(".maps");

struct event_t *e = bpf_ringbuf_reserve(&rb, sizeof(*e), 0);
if (!e) {
    __u32 k = 0;
    __u64 *d = bpf_map_lookup_elem(&drop_count, &k);
    if (d) __sync_fetch_and_add(d, 1);
    return 0;
}

```

In Go, print the total drop count alongside events received. Calculate drop percentage.

3. Mini-Project: File Integrity Monitor

Build a monitor that alerts when any process writes to /etc/:

- Hook `sys_enter_openat` tracepoint
- Filter: path contains "/etc/" AND flags include O_WRONLY/O_RDWR
- Emit event via ring buffer with: PID, UID, comm, full path, timestamp
- In Go: compare path against allowlist; print ALERT for unknown writers

4. Advanced Challenge: Throughput Benchmark

Write a Go benchmark that measures events/sec at varying ring buffer sizes:

1. Set buffer to 1MB, 4MB, 16MB, 64MB
2. Generate 1M execve events using a subprocess loop
3. Measure: events received / elapsed seconds, drop rate %
4. Plot results in a table and determine the optimal buffer size for your workload